

Rapport TP3

**Where Hidden Rhythms in Silent Vectors Lie: A Quest
into the Emergence of Structures in Character-Level
GPT Models**

**Paulette Diouf
M1 Logos**

Lien git: <https://github.com/Paulette-D/LLM.git>

INTRODUCTION:

Ce rapport présente le travail réalisé dans le cadre du lab 3, dont l'objectif est de construire, entraîner et analyser un modèle de langue autorégressif au niveau des caractères, entraîné sur un corpus de poésie française. L'objectif principal de ce projet est de ne pas traiter le modèle comme une boîte noire, mais d'en comprendre le fonctionnement interne et d'observer comment des structures linguistiques: rythme, métrique, schémas de rimes émergent progressivement au cours de l'apprentissage.

Le travail se divise en quatre étapes. La première consiste à implémenter une architecture Transformer décodeur (decoder-only), similaire aux modèles de la famille GPT, en s'appuyant sur les briques de base fournies par PyTorch. La deuxième étape couvre la constitution d'un corpus poétique et l'entraînement du modèle, accompagné d'un suivi rigoureux des métriques d'apprentissage. La troisième étape explore l'impact de plusieurs hyperparamètres sur le comportement du modèle. Enfin, la quatrième étape analyse les textes générés afin de déterminer si les propriétés structurelles du corpus d'entraînement réapparaissent dans les productions du modèle.

L'ensemble du code est disponible dans un dépôt GitHub organisé de la façon suivante :

- `codes/` : implémentation Python organisée en trois fichiers : `modele.py` (architecture complète du modèle), `entrainement.py` (constitution du corpus, boucle d'entraînement et exploration des hyperparamètres) et `analyse.py` (génération de poèmes et analyse structurelle des textes produits)
- `figures/` — ensemble des visualisations produites au cours des expériences
- `lab3_gpt_poesie.ipynb` — notebook Google Colab utilisé pour l'exécution des expériences, l'entraînement ayant été réalisé sur GPU T4

Le modèle entraîné (`modele_gpt_poesie.pt`) n'est pas versionné dans le dépôt en raison de sa taille, conformément aux règles définies dans le fichier `.gitignore`.

Difficultés rencontrées

La réalisation de ce projet a donné lieu à plusieurs difficultés pratiques. La première concerne la gestion des caractères spéciaux du corpus poétique français. Certains caractères peu fréquents, comme le œ, étaient absents du vocabulaire construit à partir du corpus, provoquant des erreurs lors de l'encodage des prompts de génération. Ce problème a été résolu en filtrant les caractères inconnus dans la fonction d'encodage.

La deuxième difficulté est liée aux contraintes de temps de calcul. L'entraînement sur CPU en local s'est révélé impraticable, la session aurait nécessité plusieurs heures. L'entraînement a donc été intégralement déporté sur Google Colab avec un GPU T4 (avec utilisation limitée). Même dans ce cadre, le nombre de steps a dû être réduit de 50 000 à 20 000 pour rester dans des délais raisonnables. De même, lors de l'exploration des hyperparamètres, le nombre de configurations testées et le nombre de steps par configuration ont dû être ajustés réduits à 5 000 steps pour éviter des temps d'exécution excessifs ainsi que la taille des données.

Enfin, l'entraînement a mis en évidence un surapprentissage marqué du modèle dès les 5 000 premiers steps, visible sur les courbes d'apprentissage. Ce phénomène, discuté en détail dans la section correspondante, est directement lié à la taille limitée du corpus et au nombre de steps d'entraînement.

Étape I: Architecture du modèle

Le modèle implémenté est un Transformer décodeur (decoder-only) opérant au niveau des caractères, ce qui permet de conserver un vocabulaire réduit et d'éviter toute tokenisation complexe.

Embeddings

Nous combinons un embedding de tokens et un embedding positionnel appris, dont la somme constitue la représentation initiale de chaque token. Nous optons pour les embeddings positionnels appris plutôt que sinusoïdaux ou RoPE pour leur simplicité et leur efficacité démontrée dans les modèles GPT originaux. La longueur de contexte retenue est $L = 256$ caractères, correspondant à environ deux à trois strophes, suffisant

pour capturer des régularités locales telles que les schémas de rimes ou la longueur des vers.

Attention causale multi-têtes

Deux implémentations ont été comparées : l'approche classique avec masque triangulaire explicite, et l'approche optimisée via `torch.nn.functional.scaled_dot_product_attention`, qui exploite *FlashAttention* automatiquement sur GPU. Cette seconde approche réduit la consommation mémoire de $O(T^2)$ à $O(T)$ et s'est révélée sensiblement plus rapide.

```
... [ classique] Temps : 4.936 ms | Mémoire GPU : 185.2 Mo
    [ optimisee] Temps : 1.796 ms | Mémoire GPU : 57.2 Mo
```

Étape II: Corpus et entraînement

Constitution du corpus

Le corpus est constitué de poésie française du domaine public, récupérée depuis le Projet Gutenberg. Les œuvres retenues sont Les Contemplations de Victor Hugo, Les Fleurs du Mal de Charles Baudelaire, les œuvres complètes de Paul Verlaine et les Poésies d'Arthur Rimbaud. Ce choix est motivé par la richesse des structures poétiques de ces œuvres, alexandrins, strophes régulières, schémas de rimes marqués. Les sauts de ligne, séparations de strophes et ponctuation ont été préservés lors du nettoyage des fichiers.

OEUVRE	CARACTÈRES
V.Hugo - Les Contemplations	254 427
Baudelaire - Les Fleurs du Mal	180 205
Verlaine - Œuvres complètes	293 773
Rimbaud - Poésies	134 494

La taille du corpus reste modeste (0.86 Mo), ce qui correspond à la catégorie expérimentation rapide définie dans le sujet. Cette contrainte, imposée par les délais,

constitue l'une des causes principales du surapprentissage observé lors de l'entraînement.

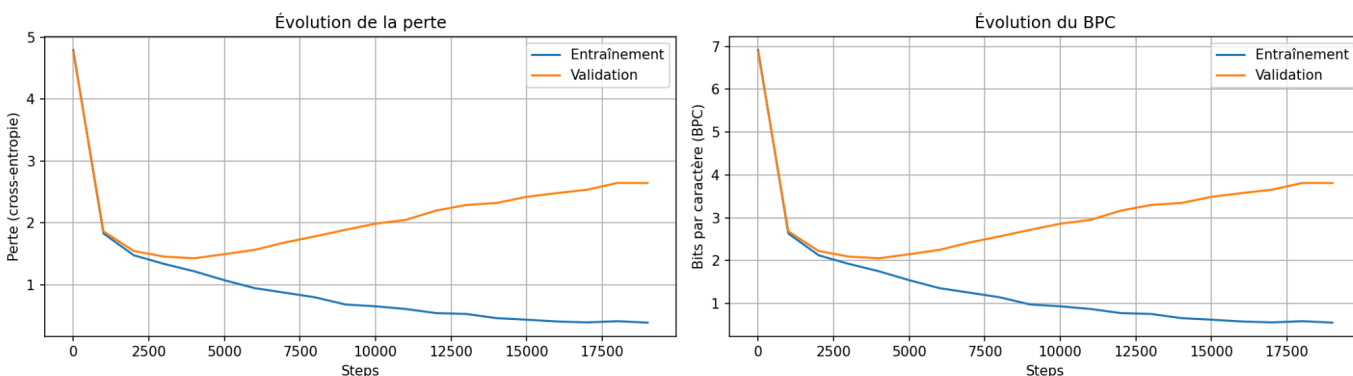
Tokenisation

Le vocabulaire est construit à partir de l'ensemble des caractères distincts du corpus.

Entraînement

Le modèle est entraîné avec l'optimiseur AdamW, un taux d'apprentissage de 3×10^{-4} , une taille de batch de 32, sur 20 000 steps avec un échauffement linéaire sur les 1 000 premiers steps et un écrêtage du gradient à 1.0.

Courbes d'apprentissage

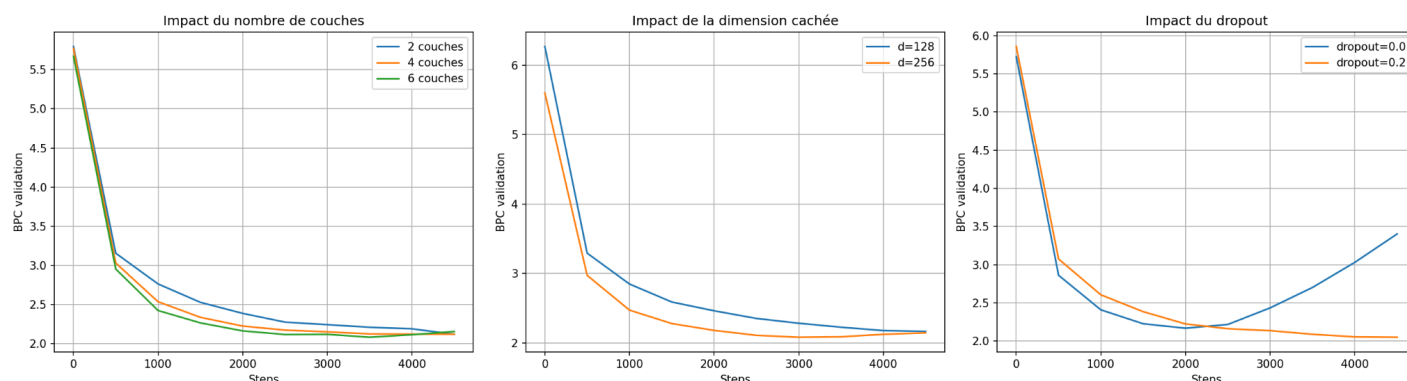


Les courbes révèlent deux phases distinctes. Durant les 5 000 premiers steps, les deux pertes descendent conjointement, indiquant que le modèle apprend des régularités généralisables. À partir de ~5 000 steps, un surapprentissage apparaît clairement : la perte d'entraînement continue de descendre pour atteindre ~0.5, tandis que la perte de validation remonte progressivement jusqu'à ~2.6. Le BPC de validation minimal, atteint autour de step 3 000-5 000 (~2.1), représente le meilleur point de généralisation du modèle. Ce surapprentissage s'explique principalement par la taille limitée du corpus, comme le confirme l'exploration des hyperparamètres dans la section suivante.

Étape III: Hyperparamètres

Trois hyperparamètres ont été étudiés : le nombre de couches, la dimension cachée et le taux de dropout. Pour chaque configuration, un modèle est entraîné sur 5 000 steps et le BPC de validation final est comparé. L'objectif n'est pas de trouver la configuration

optimale mais de comprendre l'effet de chaque paramètre sur le comportement du modèle.



1. Nombre de couches

Les trois configurations testées (2, 4 et 6 couches) convergent vers un BPC de validation similaire (~2.1) en fin d'expérience. Les différences sont faibles mais cohérentes : un modèle plus profond apprend légèrement plus vite dans les premiers steps. Ce résultat suggère que sur un corpus aussi réduit, augmenter la profondeur n'apporte pas de gain significatif, la capacité du modèle n'étant pas le facteur limitant.

2. Dimension cachée

L'impact de la dimension cachée est plus marqué. Le modèle avec d=256 converge nettement plus vite et atteint un BPC inférieur à celui avec d=128, confirmant qu'une représentation plus riche améliore la modélisation. Cependant, augmenter davantage la dimension (d=512) n'a pas été testé en raison des contraintes de temps de calcul.

3. Taux de dropout

C'est l'hyperparamètre dont l'effet est le plus visible. Sans dropout (dropout=0.0), le modèle surapprend massivement à partir de ~2 000 steps, avec un BPC de validation qui repart à la hausse jusqu'à ~3.4. Avec dropout=0.2, le modèle reste stable et continue de progresser jusqu'à la fin des 5 000 steps, atteignant un BPC de ~2.0. Ce résultat confirme directement l'observation faite sur les courbes d'apprentissage principales et justifie rétrospectivement l'utilisation d'un dropout

plus élevé pour contenir le surapprentissage sur ce corpus de petite taille.

Étape IV: Analyse des poèmes générés

Une fois le modèle entraîné, nous analysons les textes qu'il produit afin de déterminer si les propriétés structurelles du corpus d'entraînement réapparaissent dans les générations. La génération suit la procédure autorégressive standard, avec une température de 0.8 et un top-k de 40.

Exemples de poèmes générés:

```
Prompt : « Demain, dès l'aube »
=====
Demain, dès l'aube et qui l'amour malicieuse leurs beaux.

Monsieur Nialinade peuplier leur tenture usiles
Apparente les coeurs dde tout le noir contre la marquis,
Et qu'ils ont fait ce sourire, et qu'il est de la paix airs
Et que les coeurs aient peuvrent à ces branches sont ces feuilles
Où le Racin m'est pour le mo

=====
Prompt : « Les sanglots longs »
=====
Les sanglots longs dans l'histoire;
L'ombre éteints en pleurs sur la bouche;
L'aurore, frais, l'aube, et Police, ou boir,
L'ange fait sa parente au vil Paris;
Quand parmi les chères errants on voit les mots:
Telles, ils l'sont des choses à l'aile de la rîe.
Les feuillaient en prière les ailes de femmes;
Les femmes qu

=====
Prompt : « Mon enfant, ma soeur »
=====
Mon enfant, ma soeur, je vous cherche et vous.

Mai 1843.

XII

A M. FROMENT MEURICE

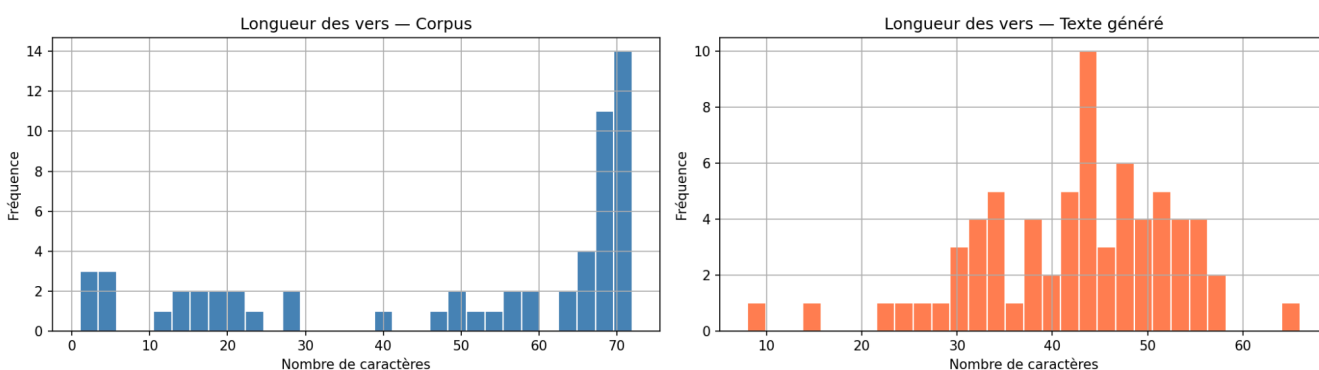
Nous sommes frères: la fleur
Par les os purs est pour jamais les roses;
Pour nous rasimerons avec les voix jamais!
C'est toute le navire ou sous les ramures,
C'est le Dieu jeune ouvrier qui s'en va chanté.
C'est bien vraiment votre

=====
Prompt : « Je suis la plaie et le couteau »
=====
Je suis la plaie et le couteau!
Car vous avez eu toutes les marâtes!
Vous êtes les mêles sont seuls noirs.
L'aigle, chante et crachant des tours;
L'amour, c'est le chien; l'aube est le jour, c'est le coeur,
Et nos bouches jetés parmi l'aimable Vibre,
Et nos parmi les douleurs sourcils et les branches,
Et sur l'horizon des voûtes
```

On observe que le modèle produit du français reconnaissable, avec une structure de vers

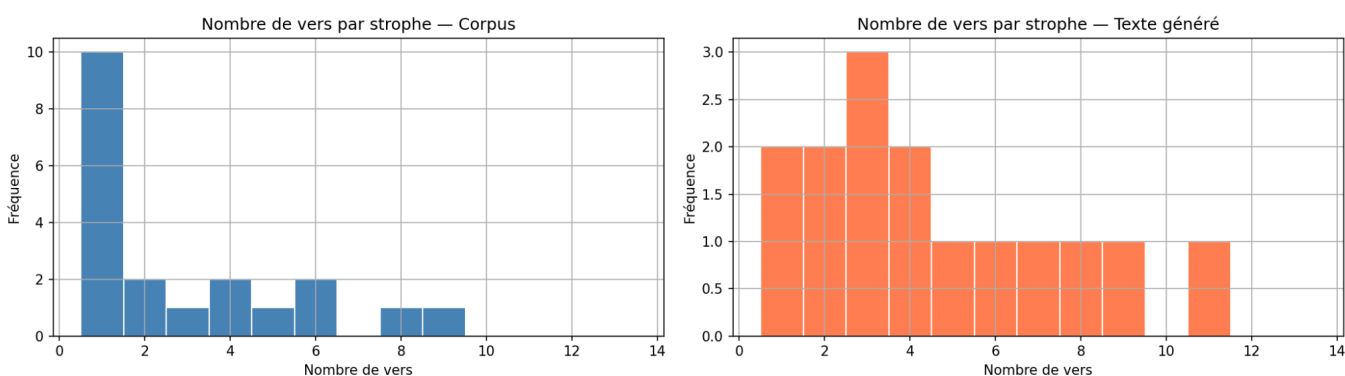
respectée et un lexique poétique cohérent avec le corpus d'entraînement. Les continuations sont stylistiquement proches des auteurs fournis en prompt, avec un usage fréquent de l'anaphore. Les erreurs restent présentes, mots inexistants, syntaxe approximative mais témoignent d'un apprentissage partiel de la structure de la langue.

Analyse de la longueur des vers



Le corpus présente une distribution des longueurs de vers concentrées autour de 60-70 caractères, reflétant la présence de longs alexandrins et de vers enrichis de ponctuation et d'indentation. Le modèle génère quant à lui des vers plus courts, centrés autour de 30-50 caractères. Cette différence s'explique par la longueur de contexte limitée à 256 caractères, qui contraint le modèle à produire des séquences plus courtes, et par le surapprentissage qui l'empêche de capturer les régularités métriques les plus fines.

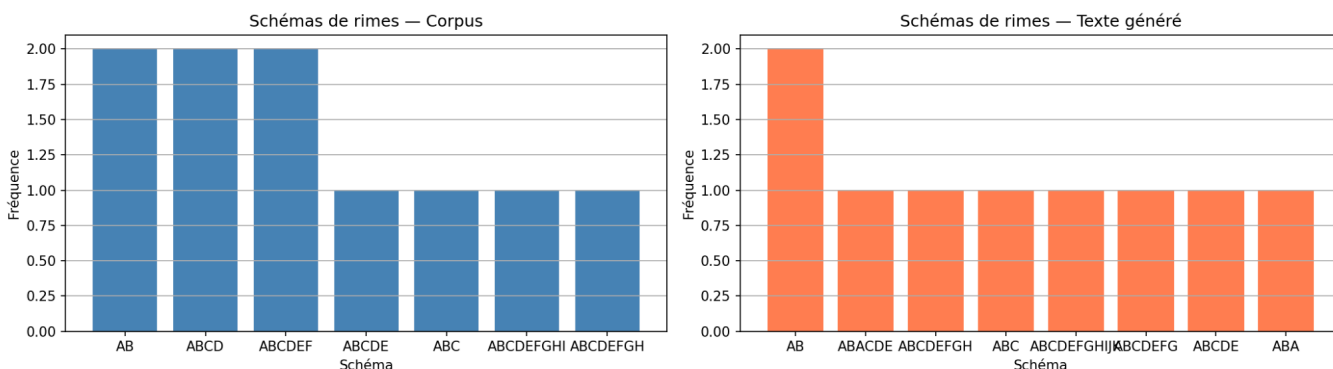
Analyse de la structure des strophes



Le corpus est dominé par des strophes d'un seul vers, résultat probable de la présence de titres et de vers isolés dans les fichiers Gutenberg. Le modèle génère en revanche des strophes de 3 à 4 vers, ce qui correspond davantage à la structure classique du quatrain.

Ce résultat montre que le modèle a peut être appris une notion de strophe plus régulière que celle présente dans le corpus brut.

Analyse des schémas de rimes



Les deux distributions partagent le schéma AB comme schéma dominant. Cependant, le corpus présente des schémas plus longs et plus variés (ABCD, ABCDEF), tandis que le modèle tend à produire des schémas courts et moins structurés. Notre méthode d'approximation des rimes, basée sur les derniers caractères des vers, reste très peu fiable et ne capte pas les véritables patterns phonétiques. Elle constitue néanmoins un indicateur de la présence de régularités de fin de vers dans les générations.

Conclusion:

Ce projet a permis de construire et d'analyser un modèle GPT au niveau des caractères, entraîné sur un corpus de poésie française. Malgré les contraintes pratiques liées à la taille du corpus (0.86 Mo) et aux ressources de calcul disponibles, le modèle parvient à produire des textes en français reconnaissables, avec une structure de vers cohérente et un lexique poétique fidèle aux auteurs du corpus.

Les expériences menées confirment plusieurs observations attendues. Le surapprentissage apparaît rapidement sur un petit corpus, et le dropout s'avère être le levier le plus efficace pour le contenir. La dimension cachée influence significativement la qualité des représentations, tandis que le nombre de couches a un impact plus limité dans ce contexte. L'implémentation optimisée de l'attention via FlashAttention s'est révélée nettement plus efficace que l'approche classique, tant en vitesse qu'en

consommation mémoire.

L'analyse des poèmes générés montre que le modèle reproduit partiellement les structures du corpus : les schémas de rimes courts, la structure en strophes de 3 à 4 vers et le lexique poétique sont bien présents. En revanche, la longueur des vers et les schémas de rimes complexes restent difficiles à reproduire fidèlement, ce qui s'explique par la taille limitée du corpus et le surapprentissage observé.

Pour aller plus loin, plusieurs pistes seraient envisageables : un corpus plus large (5-10 Mo), un early stopping basé sur la perte de validation, ou encore un dropout plus élevé (0.3) pour mieux contrôler le surapprentissage. Ces améliorations permettraient probablement d'obtenir des générations plus régulières et structurellement plus fidèles au corpus d'entraînement.